🎬 Dubbing Studio

Complete File & Folder Explanation for Code Walkthrough

A YouTube-to-Indic-Language Dubbing Pipeline

Contents

1. Project Overview & Architecture
2. Main Folder Structure
3. The Pipeline (10 Stages)
4. Core Modules
5. Server & Web API
6. Web UI (Vue 3)
7. Voice System & Models
8. Tools & Utilities
9. Key Design Decisions & Tweaks
10. Development & Testing

---

1. Project Overview & Architecture

Dubbing Studio is a complete end-to-end pipeline that takes a YouTube video link and produces a dubbed version in one of 23 Indian languages (or 46 international languages). The project is built with:

- **LangGraph** — orchestrates the 10-stage pipeline
- **FastAPI** — serves the backend API and web UI
- **Vue 3** — the frontend, no build step (served as modules)
- **PyTorch + CUDA** — GPU acceleration for models
- **Bodhan & Sarvam APIs** — translation and TTS services
- **Seed-VC** — voice cloning (speaker timbre matching)
- **LatentSync 1.6** — lip sync video generation

> **Key Concept:** Every stage outputs audio/video files and metadata (cached). The UI lets you edit the transcript in plain words ("make line 5 sadder"), and the system re-runs only the affected stages. Nothing is lost — every change creates a new version.

---

2. Main Folder Structure

Root Level Files

`README.md`

Complete documentation of the project. Contains:

- Features and distinctive capabilities
- UI highlights and design philosophy
- Hardware & software requirements
- Installation steps (4 steps)
- API keys setup (Bodhan, OpenRouter, Hugging Face, Sarvam)
- All 10 pipeline stages explained
- Command-line tools and settings
- Troubleshooting and costs

Directory Structure

`./dubbing/`

The entire application. All Python code and assets live here. Entry point for the server is `dubbing/server/__main__.py`.

`./dubbing/pipeline/`

The core dubbing pipeline. Contains the LangGraph implementation and all 10 stages (download, separate, transcribe, diarize, refine, translate, synthesize, align, lipsync, assemble).

`./dubbing/server/`

FastAPI application. Serves the web page and JSON API endpoints. Handles job queuing, SSE streaming, project management, and file exports.

`./dubbing/web/`

Vue 3 frontend application. No build step — served as .js modules by FastAPI. Contains the main app, all UI components, and the player.

./dubbing/cache/

Project storage (in-memory cache). Each project is a folder with a unique ID. Contains:

- Video/audio files
- Segment data (.json)
- Log and metadata files
- Exported outputs (MP4, M4A, subtitles)

./dubbing/cache_archive/

Archived projects (moved from cache/ when cleaned up). The library shows only 3–4 active projects; older ones are archived here to avoid clutter.

./dubbing/voices/

Bodhan voice samples and metadata. Downloaded during setup.

./dubbing/LatentSync/

Lip sync model (cloned from the LatentSync 1.6 repo during setup). Used to redraw mouth movements in the video. ~5 GB of code and trained weights.

./dubbing/SeedVC/

Voice cloning model (cloned during setup). Captures a speaker's timbre and applies it to synthesized audio. ~1 GB.

./dubbing/Separator/

Demucs vocal separator weights for extracting voice from music (for song detection and processing).

./dubbing/tests/

Unit and integration tests for critical pipeline stages. Run with `pytest`.

`./dubbing/tools/`

Command-line utilities: voice profiler, voice lister, and other dev tools.

`./scripts/`

`setup_gpu.sh` — installation script
`check_setup.sh` — post-install verification (checks models are downloaded, runs pytest)

`./backups/`

Timestamped backups of the code before major changes (e.g., `code_before_emotion_20261029_1757.tgz`). Used for rollbacks during development.

---

3. The Pipeline: 10 Stages (dubbing/pipeline/steps/)

Each stage is a node in the LangGraph graph. They read and write to `PipelineState` (defined in `state.py`).

Pipeline Variants

- **FULL_PIPELINE** — download → separate → transcribe → diarize → refine → translate → synthesize → align → lipsync → assemble (used for new dubs)
- **EDIT_PIPELINE** — translate → ... → assemble (used when the user edits the text)
- **REVOICE_PIPELINE** — synthesize → ... → assemble (used for voice casting changes)
- **LIPSYNC_PIPELINE** — lipsync → assemble (adds lip sync to fast-mode dubs later)

Stage 1: Download (dubbing/pipeline/steps/download.py)

`download_node()`

What it does: Downloads the video from YouTube (or loads a local file). Extracts the highest-quality video and audio tracks. If a clip range is specified (start/end seconds), downloads only that part.
Tools used: yt-dlp
Output: `video_path`, `audio_path` (16 kHz mono for analysis), `audio_hq_path` (44.1 kHz stereo for mixing)
Caching: If the same URL is re-run, reuses the downloaded file.

Stage 2: Separate (dubbing/pipeline/steps/separate.py)

`separate_node()`

What it does: Splits the audio into voice and background (music, effects, room tone). The background is kept and will be mixed under the dub.
Tools used: Demucs (htdemucs model)
Output: `vocals_path`, `background_path`
Design: Full instrumental background + vocals are separated; only the voice is dubbed, the rest is kept original.

Stage 3: Transcribe (dubbing/pipeline/steps/transcribe.py)

`transcribe_node()`

What it does: Converts speech to text, word-by-word with timestamps. For Indic-language videos, tries Bodhan indic-transcribe first (more accurate); falls back to Whisper large-v3 if Bodhan is not available or fails.
Tools used: Bodhan indic-transcribe (API) + faster-whisper large-v3 + Silero VAD (voice activity detection)
Output: Segments with `text`, `start`, `end`, `words` (list of word timings)
Design: VAD removes silence; Whisper runs on speech chunks only.

Stage 4: Diarize (dubbing/pipeline/steps/diarize.py)

`diarize_node()`

What it does: Identifies who spoke each line and their gender. Combines:

- **pyannote (speaker-diarization-community)** — turn detection and speaker boundaries
- **Voice embedding (ECAPA-TDR)** — speaker identity (voiceprint)
- **Face detection & gender classifier (ArcFace, wav2vec2)** — face identity and per-face gender (optional, if video has faces)
- **Prosody analysis** — delivery style (energetic, calm, surprised, etc.)

Output: Each segment gets `speaker` (ID), `gender` (male/female), `f0` (pitch), `delivery` (style), `pace` (quick/slow)
Design: Multiple signals (voice, face, audio) are combined so even crowded film scenes work correctly.

Stage 5: Refine (dubbing/pipeline/steps/refine.py)

`refine_node()`

What it does: Fixes recognition mistakes in the transcript using an LLM. Also reconciles two transcriptions (Whisper + Bodhan) if both were run. Preserves names and technical terms from the glossary.
Tools used: OpenRouter API (calls an LLM like Claude or Llama)
Output: Cleaned `text` field for each segment
Design: The glossary from the UI (names, special terms) is passed to the LLM to preserve them.

Stage 6: Translate (dubbing/pipeline/steps/translate.py)

`translate_node()`

What it does: Translates the dialogue to the target language. For Indic languages: Bodhan indic-translate (fast, grammar-aware) + LLM "dubbing director" (fixes gender agreement, spoken register, timing). For international languages: LLM does all translation.
Tools used: Bodhan indic-translate (API) + OpenRouter LLM
Output: `dubbed_text` for each segment
Design: The "dubbing director" LLM is given:

- The whole conversation (context)
- Speaker profiles (names, genders, relationships)
- Domain/style hints (formal, casual, educational)
- Timing constraints (how many words fit in the original duration)

This produces natural, flowing dialogue that fits the time available.

Stage 7: Synthesize (dubbing/pipeline/steps/synthesize.py)

`synthesize_node()`

What it does: Converts the translated text to speech in the target language. Assigns one consistent voice per speaker, matched to their gender and pitch. Uses emotion/delivery styles (excited, calm, sad, etc.).
Tools used:

- **Bodhan indic-speak** (Indic languages) — 22 languages, 2 voices each
- **Sarvam Bulbul** (optional, 11 Indic languages) — higher quality
- **Kokoro TTS** (international languages)
- **Piper TTS** (fallback)

Output: `dubbed_audio` (WAV file) for each segment
Design:

- Voice casting is done first: one voice per speaker (based on gender, pitch, user choice)
- Bodhan voice styles (happy, sad, energetic, calm, etc.) are applied per line based on the delivery/emotion
- Rate limiting: the gate reads the TTS service's response headers and obeys retry-after, so long videos don't fail

Stage 8: Align (dubbing/pipeline/steps/align.py)

`align_node()`

What it does: Fits the dubbed audio to the original speaker's timing. If a line is too fast or too slow, it's rewritten shorter/longer and re-spoken. The speech rate of each voice is measured and remembered. Then:

- Measure the dubbed audio duration
- Calculate how much stretching is needed to fit the original time slot
- If stretching > 1.15x or < 0.90x, rewrite the line (shorter or longer) and re-speak
- Stretch the audio (pitch-preserving) to fit exactly

Tools used: rubberband (pitch-preserving time stretching), OpenRouter (rewriting)
Output: `aligned_audio` for each segment (ready to mix)
Design:

- Timing constraints: MAX_STRETCH=1.10, HARD_MAX_STRETCH=1.25, MIN_RATE=0.90, GAP_GUARD=0.08s, REWRITE_ABOVE=1.15
- Words are never cut — if stretching is too much, the line is rewritten
- Per-speaker tempo: each line is allowed to say as much as the speaker normally says in that time

Stage 9: Lip Sync (dubbing/pipeline/steps/lipsync.py)

`lipsync_node()`

What it does: Redraws the mouth on video shots where the speaker's face is visible. For each frame in the video:

- Detect all faces in the frame
- Score each face with SyncNet against the original voice — only faces that were really speaking get synced
- Use LatentSync 1.6 to generate new mouth movements for those faces
- Reaction shots, photos, and wide shots are left untouched ("when unsure, don't sync")

Tools used: LatentSync 1.6 (deep learning model), RAFT (frame interpolation), SyncNet (confidence scoring)
Output: `lip_synced_clip` (MP4) for each segment
Design:

- Smart sync: only the real speaker's face is synced; listeners are untouched
- GPU requirement: ~10 GB VRAM per worker (with 24 GB GPU, 2 workers run in parallel)
- Optional: fast mode skips lip sync; `LIPSYNC_PIPELINE` adds it later

Stage 10: Assemble (dubbing/pipeline/steps/assemble.py)

`assemble_node()`

What it does: Mixes all the pieces together:

- Concatenate all the lip-synced clips (or original clips if no lip sync)
- Mix the dubbed audio track (-14 LUFS loudness)
- Mix the original background (music, effects, room tone) underneath
- Add subtitles in the dub language (and optionally the original)

Tools used: ffmpeg
Output: `final_video` (MP4), `final_audio` (M4A for browser playback)
Design: The original music and effects are preserved; only the voices change.

---

4. Core Modules (dubbing/pipeline/)

Key Files

`state.py`

Purpose: Defines the data structures passed through the pipeline.

- **Segment:** One line of dialogue. Has ~35 fields: original text, translation, audio/video files, speaker ID, gender, pitch, delivery style, emotion, voice used, lip-sync clips, etc.
- **PipelineState:** The entire project state. Has the source video, all segments, settings chosen in the UI, speaker profiles, run log, final outputs, etc.

Design: The Segment and PipelineState are immutable within a stage; each stage returns a new version with only changed fields updated.

`graph.py`

Purpose: Assembles the pipeline stages into LangGraph graphs.

- **FULL_PIPELINE:** All 10 stages in order (for new dubs)
- **EDIT_PIPELINE:** Starts from translate (when editing text)
- **REVOICE_PIPELINE:** Starts from synthesize (when changing voices)
- **LIPSYNC_PIPELINE:** Only lipsync + assemble (adds lip sync later)

`cache.py`

Purpose: Manages the project cache (where videos, audio, metadata are stored).

- Projects are stored in `dubbing/cache/<project_id>/`
- Provides functions to save/load segments, audio files, video clips, metadata JSON files
- Caching is automatic: each stage caches its output; re-running a pipeline reuses cached files for unchanged segments

5. Voice System (dubbing/pipeline/voice/)

Voice Casting & Cloning

`casting.py`

Purpose: Assigns one voice per speaker based on:

- Speaker's detected gender (male/female)
- Speaker's pitch (f0)
- User's manual choice (from the UI)

Output: `voice_map` — {"speaker_0": "Kavya_hi", "speaker_1": "Arjun_hi", ...}

`voice_clone.py`

Purpose: Clones the original speaker's timbre onto the dubbed audio using Seed-VC.

- For each speaker, use a reference audio clip (one of their original lines) to capture their unique voice characteristics
- Apply Seed-VC to all that speaker's dubbed lines so they sound like the original speaker, just speaking in the new language
- Quality checks: minimum length (0.3s), envelope correlation > 0.6, pitch within 5 semitones, gender consistency

Design: Per-line references ensure all lines from one speaker are in the same voice.

`tts.py`

Purpose: Text-to-speech synthesis.

- Supports Bodhan (22 Indic languages), Sarvam (11 languages), Kokoro, and Piper
- Each language has male and female voices
- Caches voice profiles (duration per character, rate in chars/sec)

`styles.py`

Purpose: Bodhan voice style mapping.

- Bodhan supports ~10 delivery styles: happy, sad, energetic, calm, surprised, angry, etc.

- Each line's emotion (detected from the original audio) is mapped to a Bodhan style
- The TTS output for that line uses the corresponding style

`emotion.py`

Purpose: Detects and labels the emotional delivery of each line from the original audio.

- Uses prosody features (pitch, energy, duration changes)
- Outputs emotion label: "neutral", "excited", "surprised", "sad", "calm", etc.
- Can be toggled with `EMOTION_STYLES=1` environment variable (ON by default)

`catalogue.py`

Purpose: The voice catalogue — all available voices (Bodhan, Sarvam, Kokoro, Piper) with metadata:

- Voice name, language, gender
- Which TTS provider (Bodhan/Sarvam/Kokoro/Piper)
- Voice rate (words per second)

`seedvc_worker.py`

Purpose: Wrapper around the Seed-VC model. Runs voice cloning in a subprocess to manage GPU memory and prevent model-loading conflicts.

---

6. Text & Video Processing

`dubbing/pipeline/text/`

languages.py: Language codes and names for all 23 (Indic) + 46 (international) languages.
glossary.py: Shared glossary — names and technical terms that should not be translated. Passed to the translator and cleaner.

`dubbing/pipeline/video/`

smart_sync.py: Lip sync logic. For each segment, decide which faces and frames to sync (only the speaker's face, not listeners).
latentsync_worker.py: Wrapper around LatentSync. Runs the model in a subprocess to manage GPU memory.

`dubbing/pipeline/edits/`

edit_planner.py: Parses user edits (plain English, like "make line 5 sadder") into typed actions (about 15 kinds: edit_text, change_speaker, change_delivery, toggle_keep_original, etc.). Validates each action against the project and returns a plan. Then `apply_plan()` actually executes the edits.
Design: All edits are validated and only the necessary pipeline stages re-run.

7. Server & Web API (dubbing/server/)

Main Files

`app.py`

Purpose: The FastAPI application. Defines all REST endpoints and middleware.

- **GET /api/options** — UI form options (languages, voices, styles, domains)
- **GET /api/projects** — list all projects (library)
- **GET /api/projects/{id}** — load one project's data
- **POST /api/dub** — start a new dub (YouTube link, language, etc.)
- **POST /api/projects/{id}/plan** — parse a user's edit request into a plan
- **POST /api/projects/{id}/apply** — apply the plan (re-run pipeline stages)
- **GET /api/jobs** — list running and queued jobs
- **GET /api/jobs/{id}/events** — Server-Sent Events (SSE) stream of job progress
- **GET /api/projects/{id}/versions** — version history
- **POST /api/projects/{id}/versions/{v}/restore** — restore an older version
- **GET /media/{id}/{file}** — serve video/audio/subtitles with seeking support

`jobs.py`

Purpose: Job queue and execution.

- Jobs are queued and executed one at a time (or 2 in parallel for lip sync)
- Each job is assigned a unique ID; the UI polls `/api/jobs/{id}/events` for live progress updates
- Events are SSE (Server-Sent Events) — the server pushes updates to the client in real-time

`projects.py`

Purpose: Load/save projects, metadata, and files.

- Projects are stored in `dubbing/cache/<id>/`
- Each project has a `project.json` with settings and segment data

- Supports exporting to MP4, M4A, SRT subtitles

`versions.py`

Purpose: Version history. Every change saves a new version snapshot. Users can restore any older version.

`comments.py`

Purpose: Pin comments to specific moments in the video (for collaboration).

`audition.py`

Purpose: Audition a single line: hear it in the new voice or wording before committing. Generates a short MP3 for preview.

`runner.py`

Purpose: Executes a pipeline graph. Loads the project state, runs the graph, saves results to cache.

`eta.py`

Purpose: Estimates the time for a new dub based on video length. Learns from past runs.

`startup.py`

Loads the .env file (API keys) when the server starts.

---

8. Web UI (dubbing/web/)

No build step. The UI is plain Vue 3 modules served by FastAPI. The browser loads them as ES modules and runs them directly.

`index.html`

Entry point. Loads `App.js`.

`App.vue`

Purpose: Main application component. Routes between:

- **NewDub.vue** — form to start a new dub (YouTube link, language, options)
- **DubStudio.vue** — the main editor/player (once a dub is running or loaded)
- **Library.vue** — list of all projects

`DubStudio.vue`

Purpose: The main workspace. Layout:

- **Left:** "Dub Express" (pipeline progress — 10 stations)
- **Center top:** Video player (Original / Dub / Compare, with CC)
- **Center bottom:** Transcript lines (with speaker cards, play/edit buttons)
- **Right tabs:**
  - **Ask:** Natural language edit interface ("make line 5 sadder")
  - **Lines:** Per-line editor (edit text, change speaker, change delivery style)
  - **Voices:** Speaker voice preview cards
  - **Versions:** Version history + restore
  - **Comments:** Pinned comments on the video

`components/`

Reusable UI components:

- **VideoPlayer.vue** — plays Original / Dub / Compare, with CC captions

- **StationBoard.vue** — displays one pipeline stage (station)
- **LineEditor.vue** — edit a single line (text, speaker, style)
- **LanguagePicker.vue** — Lotus flower, India map, or globe (for language selection)
- **SpeakerCard.vue** — shows a speaker's avatar, voice preview, lines

`styles/`

- **theme.css** — light/dark mode colors, Indic typography
- **global.css** — shared styles (buttons, inputs, layout)

UI Theme & Design

- **Railway-themed "Dub Express"** — the 10 pipeline steps are train stops
- **Language pickers:** Lotus flower (Indic mode), India map, or d3 globe (international mode)
- **Seamless compare** — side-by-side Original, Dub, Compare with CC subtitles
- **Light & dark themes** — Bodhan brand colors adapted for each
- **Indic-friendly typography** — Noto Sans for all Indian scripts

---

9. Key Design Decisions & Tweaks

Voice Cloning (Seed-VC)

- **Per-line references:** For each speaker, all their dubbed lines are cloned from one reference (one of their original lines) so they sound consistent.
- **Quality gates:** Reference must be at least 0.3s long, envelope correlation > 0.6, pitch within 5 semitones, gender must match.
- **Fallback:** If reference fails, the line is re-run without cloning (plain TTS).

Timing & Stretching (Align Stage)

- **Measurement before rewrite:** Measure dubbed audio duration. If stretching > 1.15x, rewrite the line shorter and re-speak instead of over-stretching.
- **Per-speaker tempo:** Each speaker has a baseline syllable rate (syllables/sec). The line is allowed to say as much as that speaker normally says in that time.
- **Timing constraints:**
  - MAX_STRETCH = 1.10 (normal limit)
  - HARD_MAX_STRETCH = 1.25 (absolute limit, logged as a warning)
  - MIN_RATE = 0.90 (minimum playback speed)
  - GAP_GUARD = 0.08s (gap between lines)
  - REWRITE_ABOVE = 1.15 (above this, rewrite instead of stretch)

Smart Lip Sync

- **SyncNet scoring:** For each face in each frame, score it against the original voice. Only faces with high confidence are synced.
- **Speaker detection:** Only the face identified as the current speaker is synced; listeners and reaction shots are left untouched.
- **Philosophy:** "When unsure, don't sync." Better to leave a shot original than to desync the lips.

Emotion Styles

- **Emotion detection:** Prosody analysis labels each line (excited, calm, sad, surprised, etc.).

- **Bodhan style mapping:** The emotion is mapped to a Bodhan delivery style; the TTS output uses that style.
- **Toggle:** Set `EMOTION_STYLES=1` (ON by default) or `EMOTION_STYLES=0` to disable.

Rate Limiting & Graceful Degradation

- **Bodhan gate:** Reads the TTS service's rate-limit response headers and waits before retrying. Long videos finish instead of failing.
- **Fallbacks:** Each model has a fallback. If Bodhan fails, try Sarvam. If Sarvam fails, try Kokoro or Piper.
- **Caching:** Every stage caches its output. Re-run a pipeline, and unchanged lines reuse cached audio.

Versions & Undo

- Every change creates a new version snapshot.
- Users can restore any older version, undo, and re-apply changes.
- Versions are JSON snapshots; no data is deleted.

Edit Validation

- When a user makes a change (plain English in the Ask tab), it's parsed into typed actions and validated against the project.
- Only the necessary pipeline stages re-run. If the user edits the wording of line 5, only line 5 re-runs synthesis, align, lip sync, and assemble.

---

10. Development & Testing

`dubbing/tests/`

Unit and integration tests. Run with `pytest`.

- **test_timing.py:** Tests for the align stage (stretching, rewriting, timing constraints). ~5 tests.
- **test_casting.py:** Tests for voice casting logic (speaker profiling, voice selection). ~4 tests.

`dubbing/tools/`

list_voices.py: CLI tool to list all available voices and their properties.
build_voice_profiles.py: Pre-compute voice rates (duration per character) and cache them.

`scripts/`

setup_gpu.sh: Installation script. Downloads dependencies, models, clones LatentSync and Seed-VC.
check_setup.sh: Post-install check. Verifies models are downloaded and tests are passing.

Environment Variables

Variable	Purpose	Default
BODHAN_TRANSLATE_KEY	Bodhan translation API key	Required
BODHAN_SPEAK_KEY	Bodhan TTS API key	Required
OPENROUTER_API_KEY	LLM API for refinement, dubbing director, edits	Required
HF_TOKEN	Hugging Face token for pyannote (speaker detection)	Optional (fallback available)
SARVAM_API_KEY	Sarvam TTS (extra voices, 11 languages)	Optional

EMOTION_STYLES	Enable emotion-based voice styles (0 or 1)	1 (ON)
STUDIO_PASSWORD	Optional password to access the studio	None

Running the Project

```
# Start the server
cd ~/raga_proj/Dubbing_Studio
source .venv/bin/activate
python -m dubbing.server

# Open the browser
http://localhost:8000

# Run tests
pytest dubbing/tests/
```

Quick Reference: File Organization

Path	Contents
`dubbing/pipeline/`	10-stage LangGraph pipeline
`dubbing/pipeline/steps/`	Each stage: download, separate, transcribe, diarize, refine, translate, synthesize, align, lipsync, assemble
`dubbing/pipeline/voice/`	Voice casting, cloning (Seed-VC), TTS, emotion styles
`dubbing/pipeline/video/`	Lip sync (LatentSync), smart sync scoring
`dubbing/server/`	FastAPI app, job queue, project management, API endpoints
`dubbing/web/`	Vue 3 UI, no build step, served as ES modules
`dubbing/cache/`	Project storage (videos, audio, metadata)
`dubbing/LatentSync/`	Lip sync model (~5 GB)
`dubbing/SeedVC/`	Voice cloning model (~1 GB)
`dubbing/tests/`	Unit tests for timing and voice casting
`scripts/`	setup_gpu.sh (install), check_setup.sh (verify)

Summary

Dubbing Studio is a modular, stage-based pipeline where:

- **Frontend:** Vue 3 (no build) + FastAPI static serving
- **Backend:** LangGraph pipeline with 10 stages, each with a fallback
- **Data:** Segments (lines of dialogue) passed through the pipeline, cached at each stage
- **Voice:** Bodhan/Sarvam/Kokoro TTS + Seed-VC cloning + emotion-based styles
- **Timing:** Smart rewriting and stretching to fit original duration
- **Lip sync:** Smart, speaker-only (SyncNet scoring)
- **Editing:** Plain-English edits parsed into actions, validated, and only necessary stages re-run
- **Versioning:** Every change is saved; users can restore older versions
- **UI:** Railway-themed Dub Express, language pickers, seamless compare, pin comments

Key tweaks & designs: per-line voice cloning, per-speaker tempo, emotion styles, SyncNet smart sync, rate-limit gate, edit validation, version snapshots, and graceful degradation with fallbacks.